Bounded Polymorphism

Our discussion on variance annotation has raised some interesting questions about type parametrization and the interplay between parametrization and subtyping.

The key to the mechanism of bounded quantification is the use of the type occurring as the upper bound to "expose" the interface that the type checker can count on to verify that certain operations make sense on all objects complying with that interface. This works well in several situations, but is not as effective with recursively defined types. Two kinds of problems arise with such types, depending upon the nature of the recursive definition, and more specifically upon its *polarity* (or *variance*).

The power of bounded polymorphism

The combination of subtyping and parametric polymorphism, mediated by *type bounds*, lets us write functions and types that are simultaneously *generic* (they work for any type satisfying a constraint) and *informative* (they know enough about that type to call its operations or preserve precise return types). Neither subtyping nor parametric polymorphism alone delivers this combination: the former gives the constraint, the latter gives the genericity, and bounds are how the two are wired together.

Upper bounds: relying on a known interface. Consider the `Shape` hierarchy we have used throughout the last lectures, with an additional abstract `area` method:

```
sealed trait Shape:
  def area: Double
  def scale(s:Int) : Shape

class Circle(val radius: Double) extends Shape:
  def area: Double = math.Pi * radius * radius
  def scale(s:Int): Circle = Circle(radius * s)

class Square(val side: Double) extends Shape:
  def area: Double = side * side
  def scale(s:Int): Square = Square(side * s)
```

Suppose we want a generic function that computes the *total* area of a sequence of shapes:

```
def totalArea[S <: Shape](shapes: List[S]): Double =
  shapes.map((s:S) => s.area).sum
```

The upper bound `S <: Shape` is what allows the function body to call `s.area`: without it, `S` would be an unknown type with no methods to call, and the call would be rejected. With the bound, the type checker knows that every element of the list has at least the operations of `Shape`, including `area`. The parameter `S`, however, remains generic: the function works equally well on `List[Circle]`, `List[Square]`, or `List[Shape]`, without forcing every caller to first widen their list to `List[Shape]`.

Preserving precise types. A related pattern concerns the precision of the returned type:

```
def first[S <: Shape](shapes: List[S]): S = shapes.head
```

The `first` function returns the first shape from a list, at the *precise* type of the list's elements. A `first(List(Circle(1.0), Circle(2.0)))` returns a `Circle` (not a `Shape`) so that the caller can then access `radius` without a downcast. The bound supplies the interface for the function body; the type parameter carries the precision back to the caller.

Lower bounds: the variance-recovery technique. Bounded polymorphism is also the mechanism we used to *rescue covariance* in the variance lecture. Recall the bounded `enqueue`:

```
class Queue[+A]:
  def enqueue[U >: A](a: U): Queue[U] = ...
```

Here the bound is a *lower* bound `U >: A`: the parameter `a` may have any type that is a supertype of `A`, and the resulting queue is widened correspondingly. Without this bound, the only way to use `enqueue` on a covariant queue would be to give up covariance altogether: the lower bound is what makes the covariant declaration sound.

Another example: autoClosed resources A further interesting case for bounded type parameters is safe resource handling:

```
def withResource[R <: AutoCloseable, T](resource: R)(use: R => T): T =
  try use(resource) finally resource.close()
```

The bound `R <: AutoCloseable` does two things. First, it *restricts* which types may be plugged in as `R`: only subtypes of `AutoCloseable`, such as `FileInputStream`, `java.sql.Connection`, or `Scanner`. Second, and more importantly, it *enables* the body: the call `resource.close()` typechecks precisely because the bound guarantees that every `R` we accept has a `close()` method.

A typical use:

```
val firstLine = withResource(new Scanner(new File("data.txt"))) { sc =>
  if sc.hasNextLine then sc.nextLine() else ""
}
// Scanner is closed automatically, even if reading throws.
```

Shortcomings of bounded polymorphism

Bounded polymorphism is aone of the key mechanism that makes Scala's type system both flexible and safe. It lets generic code rely on specific interfaces (upper bounds), and it lets variance discipline coexist with operations that would otherwise force invariance (lower bounds).

Below, we examine situations where this mechanism, powerful as it is, runs into structural limits. These are situations involving *recursive* parametric definitions, where bounded quantification alone does not provide what we need.

Shapes and positive recursion. We start with positive recursion, showing that in such cases, bounded quantification doe not ensure the best possible typing accuracy.

Given our `Shape` hierarchy, consider the following function to scale shapes of a factor 2.

```
def scale2x(s : Shape): Shape = s.scale(2.0)
```

This function works nicely on arguments of any subtype of `Shape`, (e.g. `Circle`), but it is not really what we are looking for,

```
scale2x(Circle(2.0)) : Shape
```

which means that we have lost the information that the result of scaling a `Circle` is still a `Circle`.

Ideally we would like to have a polymorphic function that does not simply work on subtypes of shapes (like `area`), but also returns shapes of the appropriate type.

However, bounded polymorphism, in the incarnation we are discussing, does not help:

```
def scale2x[U <: Shape](s : U) : U = s.scale(2.0)
-- Type Mismatch Error: ------------------------------------------------
|     def scale2x[U <: Shape](s : U) : U = s.scale(2.0)
|                                          ^^^^^^^^^^^^
|            Found:    Shape
|            Required: U
|            where:    U is a type in method scale2x with bounds <: Shape
```

The error is the result of the fact that the only information available to type-check the body of the function is that the parameter `s` has a method `scale`, whose return type is `Shape`.

Comparables and negative recursion. The situation is even worse with negative recursion. Consider the `Comparable` class and its subclass `Rational` discussed in the lecture on subtyping. To overcome the problems we signaled there, we may try the following enhanced `Comparable` definition with bounded polymorphism:

```
trait Comparable:
  def compare[U <: Comparable](that: U) : Boolean
```

Unfortunately, this falls short of providing a solution, as the same problem with the implementation of `Rational` resurfaces here:

```
class Rational(val num: Int, val den: Int) extends Comparable:
  def compare[U <: Rational](that: U): Boolean =
    this.num * that.den < that.num * this.den
// -- Error: ----------------------------------------------------------------
//   |  def compare[U <: Rational](that: U): Boolean =
//   |                ^^^^^^^^^^
//   |       Type bound U <: Rational does not conform to U <: Comparable
//   |       from the overridden method
```

This is a direct conseguence of sutype-checking. The trait promises that compare accepts any `U <: Comparable`, every subtype of `Comparable`, not just `Rational`. This is the function-subtyping rule applied to type parameters: just as a method's parameter type can only be widened in an override (contravariance), a type bound for a variable that occurs in contravarian position can only be widened.

F-Bounded Polymorphism

The previous attempts failed for a structural reason, not a syntactic one: a trait that talks about *itself* — like `Comparable`, whose `compare` method takes another `Comparable` — cannot express the constraint we actually want, namely *"I can be compared with values of my own concrete type"*. Writing `compare(that: Comparable)` is too loose (a `Rational` ends up comparable with any other `Comparable`, losing access to `num` and `den` without a cast), while writing `compare(that: Rational)` in the subclass is rejected by the type checker because method parameters are invariant under overriding.

The standard fix is to **abstract the trait over the concrete subtype**, turning `Comparable` into a type constructor parameterized by the type involved in the comparison.

```
trait Comparable[A]:
  def compare(that: A): Int
```

Now, we introduce `Rational` as an extension of `Comparable[Rational]`, tying the parameter of `compare` to `Rational` itself, so the implementation can access `that.num` and `that.den` directly.

```
case class Rational(num: Int, den: Int) extends Comparable[Rational]:
  def compare(that: Rational): Int = num * that.den - that.num * den
```

Interestingly, we can show that `Rational <: Comparable[Rational]` which is why the type checker acceptes the `extends` declaration:

$$\begin{aligned} \mathsf{Rational} &= \{\mathsf{num} : \mathsf{Int}, \mathsf{den} : \mathsf{Int},\ \mathsf{compare} : \mathsf{Rational} \to \mathsf{Int}\} \\ &<: \{\mathsf{compare} : \mathsf{Rational} \to \mathsf{Int}\} \\ &= \mathsf{Comparable}\{\mathsf{Rational}\} \end{aligned}$$

The advantage of F-bounded polymorphism becomes sharper when the abstract operation must *return* a value of the same concrete type, not just consume one.

Parameterizing `Shape` over its concrete subtype recovers the precision that was lost with the recursive definitions

```
trait Shape[A]:
  def scale(factor: Double): A

case class Circle(radius: Double) extends Shape[Circle]:
  def scale(factor: Double): Circle = Circle(radius * factor)
```

Now, `Circle(1.0).scale(2.0)` now has static type `Circle`, so `.radius` is accessible without pattern matching — the type system carries forward the information we always had informally.

This is the same shape of solution as `Comparable[A]`, applied to a setting where the payoff is even more visible: F-bounded polymorphism lets an abstract interface name "my own concrete type" in both argument *and* return position, preserving precise types through chains of operations.

Again, the declaration typechecks, because consistent with the corresponding subtype relationship.

$$
\begin{aligned}
\text{Circle} &= \{\text{radius} : \text{Double},\ \text{scale} : \text{Int} \to \text{Circle}\} \\
&<: \{\text{scale} : \text{Int} \to \text{Circle}\} \\
&= \text{Shape}\{\text{Circle}\}
\end{aligned}
$$

Now we can we can rewrite `scale2x` as the following polymorphic function.

```
scala> def scale2x[A <: Shape[A]](s : A) = s.scale(2.0)
scale2x[A <: Shape[A]](s: A): A
```

Note that the type bound has the same `A <: F[A]` structure as the `extends` declarations above, tying `A` withing the type constructor `F`.

This pattern is called **F-bounded polymorphism** (the *F* comes from *functor*, or type-level constructor used in the bound).

The new version of `scale2x`

- typechecks, as the upper bound `Shape[A]` exposes enough information to verify the the invocation `s.scala(2.0)`;
- returns the precise type `A`;
- may legally be instantiated to `Circle`, because `Circle` satisfies the F-bound `Circle <: Shape[Circle]` (and, when invoked on circles returns circles as desired)

F-bounded polymorphism at work

We illustrate F-bounded polymorhism on a furhter example, involving cloning.

```
trait Cloneable[A]:
  def clone(): A

def duplicate[A <: Cloneable[A]](x: A): (A, A) =
  (x, x.clone())
```

The trait `Cloneable[A]` states that cloning produces an `A`. The bound `A <: Cloneable[A]` on `duplicate` admits only types that know how to clone themselves *and* return themselves. Inside the body, `x.clone()` has type `A` — not `Cloneable[A]` — precisely because of the recursion in the bound. This is where the extra precision of F-bounds, over ordinary upper bounds, becomes visible.

A concrete use:

```
case class Document(content: String) extends Cloneable[Document]:
  def clone(): Document = Document(content)

val (original, copy) = duplicate(Document("hello"))
// both have type Document, not Cloneable[Document]
```

Compare this to what would happen with a plain upper bound `A <: Cloneable[_]`: the call `x.clone()` would yield a `Cloneable[_]`, and we would have lost track of the fact that the result is itself a `Document`. The recursive bound is what closes the loop.

Concluding remarks on subtyping and parametrization

Now that we have discussed a variety of concepts and mechanisms, related to subtyping and parametrization, it is worth stepping back to organize the broader picture. Whenever subtyping and type parameterization meet, three orthogonal questions arise, and conflating them is a common source of confusion. We illustrate them below.

1. Variance of the parameter: *Given a type constructor `F[_]`, and two types `A <: B`, how do `F[A]` and `F[B]` relate?*

- *Covariant* `F[+A]`: `A <: B` ⟹ `F[A] <: F[B]`. Example: `List[+A]` in Scala — `List[Circle] <: List[Shape]`, because an immutable list of ccircles ats is "readable as" a list of shapes.
- *Contravariant* `F[-A]`: `A <: B` ⟹ `F[B] <: F[A]`. Example: `Function1[-A, Double]` in its input parameter — `Shape => Double <: Circle => Double`, because a function accepting any shape can be used wherever one accepting circles is required.
- *Invariant* `F[A]`: no relation. Example: `Array[A]` in Scala, necessary when `F[A]` is both read from and written to.

Variance is a *design* choice of the constructor, constrained by where `A` appears (positive vs. negative positions). F-bounds enter here only marginally: `Comparable[-A]` would be the natural variance choice (`A` appears only in input position), but in practice, with F-bounded polymorphism one tends to keep things invariant, because the self-reference muddles the interpretation.

2. Type bounds: *which types can I plug in as the parameter of a polymorphic function?*

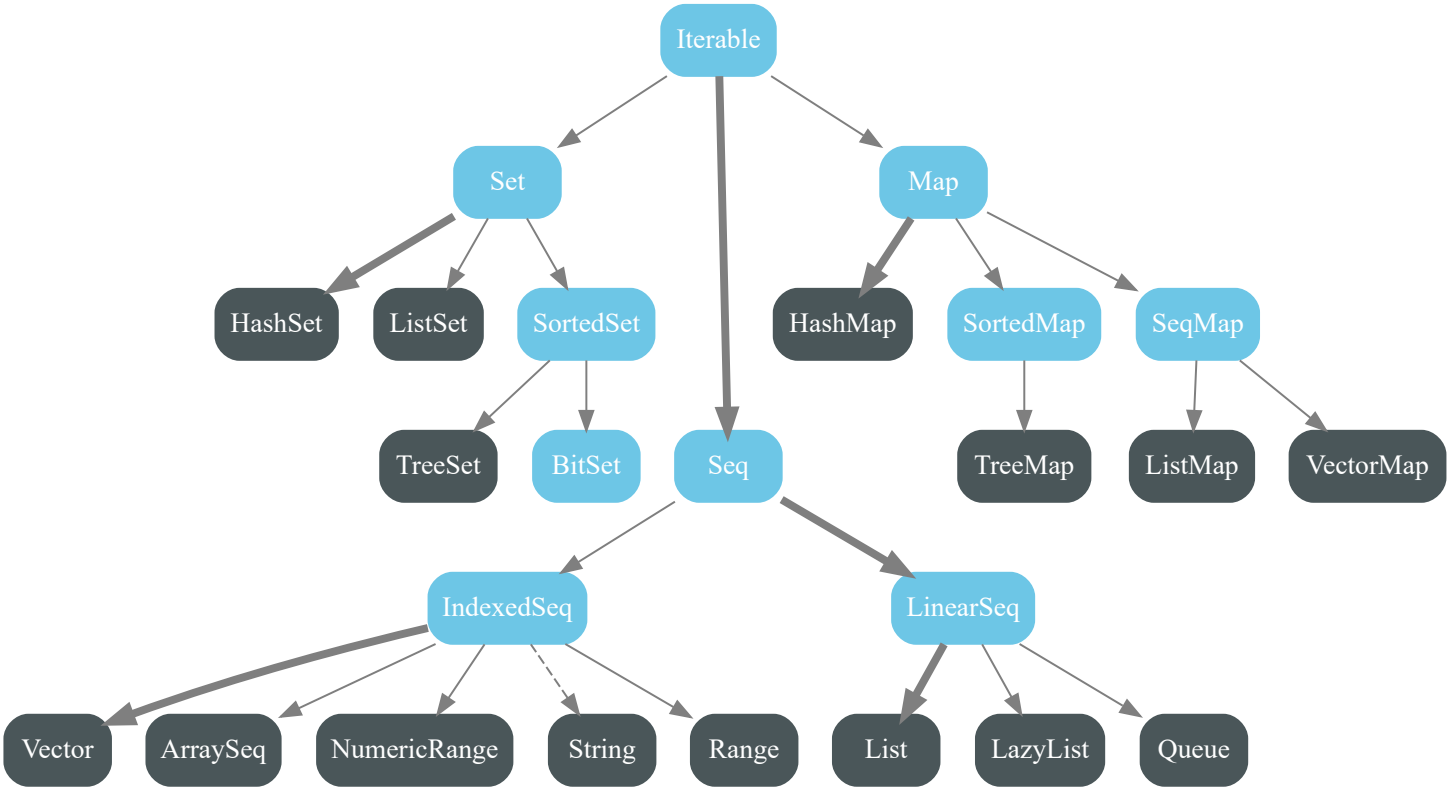
With no bound, a function `def f[A](x: A): ...` accepts any `A`, but the body can do almost nothing with `x` — the parameter is completely opaque. An *upper bound* `def f[A <: U](x: A): ...` admits only subtypes of a fixed `U`, and in exchange the body may call any operation available on `U`. The bound thus plays a double role: it *restricts* which types the caller may supply, and it *enables* the body to use the operations granted by the bound. Lower bounds `A >: L` exist too but are much rarer, mostly used to relax variance constraints (for example `def prepend[B >: A](x: B): List[B]` in a covariant `List`).

The recursive case is **F-bounded polymorphism**: `def f[A <: G[A]](x: A): ...`, where the bound itself is a type-level function of `A`. This is the same mechanism as an ordinary upper bound, only with the bound depending on `A`, and it is exactly what makes self-referential abstractions like `Comparable[A]` and `Shape[A]` precise — letting a function not just *call* operations on its argument, but also *return* a value of the same concrete type. Bounds and variance compose freely: a polymorphic function may declare a parameter with both an annotation and a bound, though variance most often appears on the type parameters of containers rather than of functions.

3. Relationship between constuctors: Fixing the parameter, when is `F[A]` <: `G[A]` for two distinct constructors `F[_]` and `G[_]`

Here, the answer is governed not by variance or bounds, but by the declared subtype hierarchy, and correspondingly by the associated structurel sutyping rules validating that the substitution principle holds between the operations of `F` and `G`.

The Scala collections library is an interesting case study.



All the nodes in the tree are parametrized types, and each edge denotes `extend` relationships relationship from the lower node to the upper one.

Each edge in this diagram is a deliberate design choice, justified by the structural subtyping rules: for instance, `List[A] <: Seq[A] <: Iterable[A]` holds because each level adds operations or strengthens guarantees without invalidating those above it.

On the other hand `List[A]` and `Set[A]` are siblings, since Set's contract (uniqueness, idempotence of insertion) is one that List does not satisfy. L

References and Credits

The material on this lecture is based on the seminal paper on the F-Bounded Polymorphism by P. Canning *et al.*